

AdaptLearn: AI-Powered Adaptive Learning Portal

Working title. Rename freely. Built for the FACE Prep Campus *Full Stack Web Development* skill-task sheet (15 tasks). Every feature in the brief is kept as-is; the AI/LLM layer is added on top as a separate service.

1. Overview

AdaptLearn is a full-stack learning portal where students enrol in courses built from free NPTEL lecture videos (embedded from YouTube), learn on a single lesson page, and receive a **personalised study plan** driven by their actual performance: watch behaviour, quiz results and topic mastery. An AI tutor answers questions grounded in the course material, and administrators get progress analytics and AI-assisted course authoring.

One-line pitch: *NPTEL content + progress tracking + a RAG tutor + an adaptive study planner, delivered as a deployed MERN application with a Python AI microservice.*

2. Problem statement

Students struggle to:

- find suitable courses and enrol efficiently,
- keep course videos, notes and quizzes in one place,
- monitor real progress instead of just "videos opened",
- know **what to study next** and what to revisit,
- get doubts answered while they are watching a lecture.

Administrators struggle to:

- manage courses, students and enrolments,
- see which students or topics are falling behind,
- author structured course content quickly.

Free, high-quality content (NPTEL) already exists, but it has no feedback loop. AdaptLearn adds that loop.

3. Target users and requirements

User	Key requirements
Student	Register/login securely, browse and search courses, enrol, watch lessons in-page, take quizzes, see progress and mastery, get a weekly study plan, ask the tutor questions, receive notifications, chat with classmates
Administrator	Secure login, add/edit/delete courses and students, import an NPTEL playlist as a course, review AI-generated quizzes before publishing, monitor enrolments, view analytics and at-risk students, generate reports

4. Features

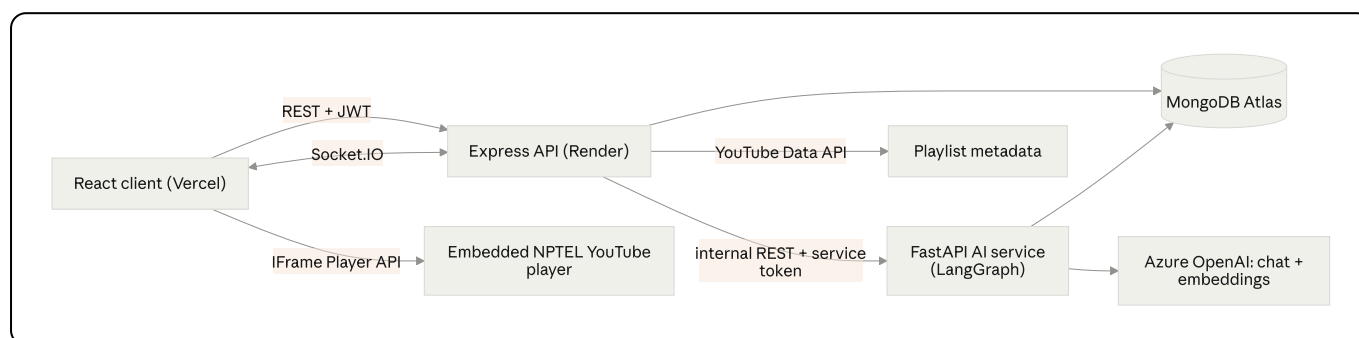
4.1 Core features (from the FSWD brief, unchanged)

Feature	Brief task(s)
Registration, login, logout with client-side validation	2, 4, 5
Dynamic navigation based on login state	4
Student dashboard (enrolled, completed, ongoing, progress stats)	2, 7
Admin dashboard (students, courses, active enrolments, completion reports)	7, 9
Course management: add, view, update, delete	9, 12
Course browsing, search, enrolment	9, 12
Learning modules, materials, mark module complete	7, 12
Progress tracking: completion %, completed/pending modules	7, 12
JWT authentication, protected routes, 401 handling	10
Real-time notifications and student chat (Socket.IO)	13
Password hashing, validation, CORS, centralised errors, logging	14
Deployment, testing, live URLs	15

4.2 AI features (added on top)

ID	Feature	What it does
A1	RAG tutor	Answers questions from the enrolled course's transcripts/notes, cites lesson and timestamp, refuses when the material does not cover the question. Replies stream token-by-token over Socket.IO.
A2	Quiz generation	Creates per-lesson quizzes (structured JSON, schema-validated). Admin reviews before publishing.
A3	Adaptive study planner	Computes topic mastery from performance signals, then produces a weekly plan: what to revisit (with video timestamps), what to practise, what to do next.
A4	Course Studio (admin)	Paste an NPTEL playlist URL: the system imports videos, groups them into modules, tags topics and drafts quizzes.
A5	Admin analyst	Natural-language questions ("which topic has the lowest mastery this month?") answered through a fixed set of safe aggregation tools, plus at-risk student flags and generated reports.
A6	Smart notifications	Nudges and weekly digests generated from progress data, delivered in real time.

5. System architecture



Why this shape

- **Express stays the single public backend.** All the brief's requirements (REST APIs, JWT, MongoDB CRUD, Socket.IO, security) live here and remain visible to reviewers.
- **The AI service is internal.** The browser never talks to it. Express authenticates the user, then forwards the request with the user's identity and role.
- **Mastery is computed in code, not by the LLM.** The LLM explains and structures; deterministic logic decides. This makes the planner testable and explainable.
- **Videos are embedded, not hosted or scraped.** The YouTube Iframe Player API gives watch position, duration and play/pause/end events, which are the performance

signals.

Example flow: student asks the tutor a question

1. Student pauses at 14:20 of Lecture 4 and types a question.
2. Client emits `tutor:ask` (question, lessonId, timestamp) over Socket.IO with the JWT.
3. Express verifies the token, checks enrolment, forwards to `POST /ai/tutor` on the FastAPI service.
4. LangGraph: guard input, retrieve top chunks (Atlas Vector Search, filtered to this course), grade relevance, then generate a grounded answer with citations, or refuse.
5. Tokens stream back via Express to the client as `tutor:token`, then `tutor:done` with citations (lesson + timestamp).
6. Message, retrieved chunk IDs, latency and token cost are stored for evaluation.

Example flow: study plan

1. Client opens `/plan`. Express calls `GET /ai/plan/:courseId`.
 2. Mastery service reads `topic_mastery`, `quiz_attempts`, `watch_events`.
 3. Deterministic rules choose revisit / practise / advance items and fit them into the student's weekly hours.
 4. LLM writes the rationale in plain language and returns schema-validated JSON.
 5. Plan is stored in `study_plans` and a notification is pushed.
-

6. Tech stack

Layer	Technology
Static frontend (Tasks 2-5)	HTML5, CSS3 (Flexbox/Grid, media queries), vanilla JavaScript with ES6 modules
React frontend (Tasks 6-7+)	React + Vite, React Router, Context API (or Redux Toolkit), Axios, Framer Motion, YouTube IFrame Player API
Mock backend (Task 7)	Mock Service Worker (MSW) or JSON Server
Backend	Node.js, Express, Mongoose, Socket.IO
Auth and security	JWT, bcrypt, helmet, express-rate-limit, zod or express-validator, CORS
Database	MongoDB Atlas (documents plus Atlas Vector Search for embeddings)
AI service	Python, FastAPI, LangGraph, Pydantic, Azure OpenAI (chat and embeddings)
Video and content	Embedded NPTEL YouTube videos, YouTube Data API v3 for playlist metadata, NPTEL-provided transcripts/notes or admin-uploaded material
Testing	Postman, Jest + Supertest (API), Vitest + React Testing Library (UI), pytest (AI service)
DevOps	Git/GitHub, GitHub Actions CI, Docker, Vercel (frontend), Render or Azure Container Apps (backends)

7. Repository structure

```
adaptlearn/
├─ README.md
├─ PROJECT_PLAN.md
├─ docker-compose.yml
├─ .env.example
├─ .github/workflows/ci.yml
├─ docs/
│   └─ architecture.md
│   └─ er-diagram.md
│   └─ api-spec.md
│   └─ evaluation-results.md
│   └─ screenshots/
├─ postman/
│   └─ adaptlearn.postman_collection.json
│
├─ frontend-static/                                # Tasks 2-5 (plain HTML/CSS/JS)
│   └─ index.html
│   └─ login.html
│   └─ register.html
│   └─ dashboard.html
│   └─ lesson.html                                # placeholder embed
│   └─ css/ (base.css, layout.css, components.css, responsive.css)
│   └─ js/
│       └─ auth.js
│       └─ api.js
│       └─ validation.js
│       └─ ui.js
│       └─ state.js
│       └─ main.js
│
├─ client/                                          # Tasks 6-7+ (React)
│   └─ index.html
│   └─ vite.config.js
│   └─ src/
│       └─ main.jsx
│       └─ App.jsx
│       └─ api/                                    # axios instance, auth.api.js,
courses.api.js, ai.api.js
│       └─ mocks/                                # MSW handlers (incl. /api/ai/*)
│       └─ context/                              # AuthContext, CourseContext, AIContext,
SocketContext
│       └─ routes/                              # AppRoutes.jsx, ProtectedRoute.jsx,
AdminRoute.jsx
```

```

|       └─ hooks/                # useAuth, useYouTubePlayer,
useWatchTracker, useTutorStream
|       └─ components/
|       |   └─ layout/           # Navbar, Footer, Sidebar
|       |   └─ common/          # Button, Card, Loader, ErrorBanner,
ProgressBar
|       |   └─ courses/         # CourseCard, CourseList, SearchBar
|       |   └─ learning/        # VideoPlayer, LessonList, QuizCard,
NotesPanel
|       |   └─ ai/              # ChatTutor, StudyPlanCard,
RecommendationList
|       |   └─ admin/           # PlaylistImporter, QuizReviewTable,
AtRiskTable
|       └─ pages/
|       |   └─ Home.jsx, Login.jsx, Register.jsx
|       |   └─ Dashboard.jsx, Courses.jsx, CourseDetail.jsx
|       |   └─ Lesson.jsx, StudyPlan.jsx, Chat.jsx, Notifications.jsx
|       |   └─ admin/ (AdminDashboard, ManageCourses, ManageStudents,
CourseStudio, Reports)
|       └─ styles/
|       └─ utils/                # validators.js, formatters.js
|
└─ server/                      # Tasks 8-14 (Express)
|   └─ package.json
|   └─ src/
|       └─ server.js            # entry point (HTTP + Socket.IO)
|       └─ app.js              # middleware, routes
|       └─ config/             # env.js, db.js, cors.js
|       └─ routes/             # auth, students, courses, lessons,
enrollments, progress, quizzes, ai, notifications
|       └─ controllers/        # thin: parse request, call service,
send response
|       └─ services/           # business logic, aiClient.js (calls
FastAPI), youtube.service.js
|       └─ models/             # User, Course, Module, Lesson,
Enrollment, Progress, WatchEvent, Quiz, QuizAttempt, TopicMastery, StudyPlan,
Notification, ChatMessage
|       └─ middleware/         # auth.js (JWT), role.js, validate.js,
errorHandler.js, rateLimit.js, logger.js
|       └─ sockets/            # index.js, tutor.socket.js,
chat.socket.js, notify.socket.js
|       └─ validators/         # zod/express-validator schemas
|       └─ utils/              # ApiError.js, asyncHandler.js
|   └─ tests/
|

```

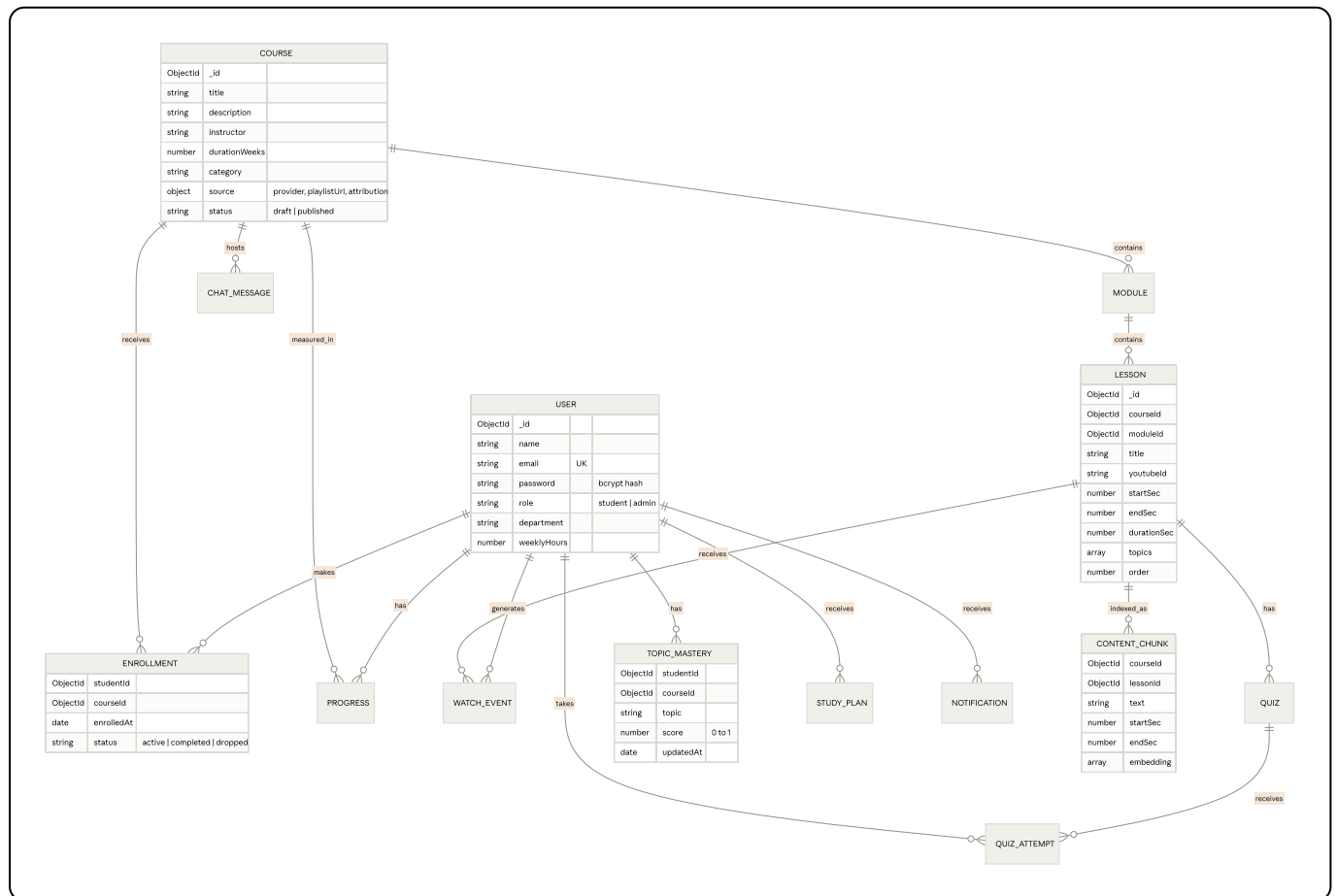
```

└─ ai-service/                                # AI/LLM microservice (FastAPI +
LangGraph)
  └─ requirements.txt
  └─ Dockerfile
  └─ app/
    └─ main.py
    └─ config.py
    └─ api/                                # tutor.py, plan.py, quiz.py, studio.py,
admin.py, health.py
    └─ graphs/                            # tutor_graph.py, quiz_graph.py,
planner_graph.py, studio_graph.py
    └─ services/                          # llm.py (provider interface),
embeddings.py, retrieval.py, mastery.py, planner_rules.py, ingest.py
    └─ guardrails/                        # input_guard.py, output_guard.py,
pii.py
    └─ schemas/                           # Pydantic models for every structured
LLM output
    └─ prompts/                           # versioned prompt templates
    └─ auth.py                            # verifies service token and forwarded
user claims
    └─ eval/
      └─ dataset.jsonl                    # 40 tutor questions (30 answerable, 10
unanswerable)
      └─ run_eval.py
      └─ results/
    └─ tests/

```

Keep the static site (`frontend-static/`) in the repo as the Tasks 2-5 deliverable, even after the React app replaces it.

8. Data model



Notes

- The brief lists both *Students* and *Users*. Use **one** `users` collection with a `role` field and student profile fields (department, studentId). Simpler auth, same outcome.
- Unique indexes: `users.email`, `enrollments (studentId, courseId)`, `progress (studentId, courseId)`, `topic_mastery (studentId, courseId, topic)`.
- `watch_events` are written in batches from the client (for example every 15 seconds and on pause/end) to avoid flooding the API. Add a TTL index if storage becomes a concern.
- `content_chunks` carry a vector index (Atlas Vector Search) with a pre-filter on `courseId`.
- Supporting collections: `modules`, `progress`, `watch_events`, `quizzes`, `quiz_attempts`, `study_plans`, `notifications`, `chat_messages`.

9. API reference

Base path `/api`. All routes except auth, health and the home route require a JWT.

Area	Method and path	Access
Status	<code>GET /</code> server status; <code>GET /api/health</code> API health	public
Auth	<code>POST /api/auth/register</code> , <code>POST /api/auth/login</code> , <code>GET /api/auth/me</code> , <code>POST /api/auth/forgot-password</code>	public / user
Students	<code>GET /api/students</code> , <code>GET /api/students/:id</code> , <code>PUT /api/students/:id</code> , <code>DELETE /api/students/:id</code>	admin (own profile for student)
Courses	<code>GET /api/courses</code> , <code>GET /api/courses/:id</code> , <code>POST /api/courses</code> , <code>PUT /api/courses/:id</code> , <code>DELETE /api/courses/:id</code>	read: user, write: admin
Lessons	<code>GET /api/courses/:id/lessons</code> , <code>GET /api/lessons/:id</code>	enrolled user
Enrolments	<code>POST /api/enrollments</code> , <code>GET /api/enrollments/me</code>	student
Progress	<code>POST /api/progress/watch</code> (batched events), <code>POST /api/progress/lessons/:id/complete</code> , <code>GET /api/progress/me</code>	student
Quizzes	<code>GET /api/lessons/:id/quiz</code> , <code>POST /api/quizzes/:id/attempts</code>	student
AI	<code>GET /api/ai/plan/:courseId</code> , <code>POST /api/ai/plan/:courseId/regenerate</code>	student
AI (admin)	<code>POST /api/ai/studio/import</code> , <code>POST /api/ai/quiz/generate</code> , <code>POST /api/ai/admin/report</code>	admin
Notifications	<code>GET /api/notifications</code> , <code>PATCH /api/notifications/:id/read</code>	user

Socket.IO events (authenticated with the JWT in the handshake)

Event	Direction	Purpose
<code>notification:new</code>	server to client	real-time alerts and nudges
<code>course:join</code> , <code>chat:message</code>	both	course discussion chat between students
<code>presence:update</code>	server to client	online/offline status
<code>tutor:ask</code>	client to server	question with lessonId and timestamp
<code>tutor:token</code> , <code>tutor:done</code> , <code>tutor:error</code>	server to client	streamed answer, final citations, failures

Response conventions: JSON only; `200/201` success, `400` validation error, `401` missing/invalid token, `403` wrong role, `404` not found, `409` duplicate, `429` rate limited, `500` handled server error with a safe message.

10. AI design

10.1 Graphs (LangGraph)

Graph	Nodes
Tutor	<code>guard_input</code> → <code>retrieve</code> (vector search, filtered to course) → <code>grade_relevance</code> → <code>generate_answer</code> (with citations) or <code>refuse</code> → <code>guard_output</code>
Quiz	<code>select_chunks</code> → <code>generate_questions</code> (JSON schema) → <code>validate</code> (schema, one correct answer, answer supported by source text) → <code>store_as_draft</code>
Planner	<code>load_signals</code> → <code>compute_mastery</code> (code) → <code>select_items</code> (rules, code) → <code>explain_plan</code> (LLM) → <code>validate_schema</code>
Studio	<code>fetch_playlist</code> → <code>group_into_modules</code> → <code>tag_topics</code> → <code>ingest_transcripts</code> (chunk, embed, index) → <code>draft_quizzes</code>

10.2 Mastery model (starting values, tune with real data)

For each topic `t` in a course:

- `q` = latest quiz score for the topic (0-1)
- `w` = fraction of the topic's lessons watched to at least 90%

- $r = \text{retry penalty} = \min(0.2, 0.05 \times \text{retries})$
- $\text{raw} = \text{clamp}(0.6 \cdot q + 0.3 \cdot w - r, 0, 1)$; if no quiz yet, $\text{raw} = \min(0.5, w)$
- $\text{mastery}_t = 0.7 \cdot \text{raw} + 0.3 \cdot \text{previous_mastery}$ (exponential moving average)

Plan rules:

Mastery Action

below 0.6 **Revisit:** link the lesson with the timestamp range tied to the weak topic, then re-quiz

0.6 to 0.8 **Practise:** extra quiz questions on the topic

above 0.8 **Advance:** next lesson

Items are packed into the student's weeklyHours using lesson durations. The LLM only writes the explanation ("You scored well on joins but missed normalisation, so start with Lecture 4 from 12:30").

10.3 Guardrails

- Input length limits and rate limits per user; token budget per request.
- **Prompt-injection defence:** retrieved text is passed as quoted data, never as instructions; the system prompt is fixed server-side; the answer must cite retrieved chunks or refuse.
- **Refusal rule:** if the best retrieval score is below a threshold, say the material does not cover it.
- Every structured LLM output is validated against a Pydantic schema, with one retry on failure.
- The admin analyst can only call whitelisted aggregation functions. The LLM **never** writes raw database queries.
- Redact emails and phone numbers before logging prompts.
- Log per request: latency, tokens, estimated cost, retrieved chunk IDs, refusal or not.
- The LLM provider sits behind one interface (llm.py) so it can be swapped if quota or access is a problem.

10.4 Evaluation (report results in the README)

What	How
Tutor answers	40 questions: 30 answerable, 10 not covered by the material. Measure correctness, citation accuracy, and refusal accuracy on the 10
Retrieval	Hit@k: is the right chunk in the top k?
Quiz quality	Human review of a sample: valid, unambiguous, answer supported by the source
Planner	Unit tests on the deterministic logic (mastery updates, thresholds, hour packing)
Performance	p50 and p95 latency, cost per chat message

Run `eval/run_eval.py` in CI on a small subset and publish the full results table in `docs/evaluation-results.md`.

11. Roadmap: the 15 tasks, in order

The AI service is built **after the database is working** (Task 12) and before Tasks 13–14 need it. Until then, the frontend uses **mock AI endpoints** (Task 7), so the UI can be finished early.

#	Brief task	What you build	AI addition	Done when
1	GitHub repo and docs	Repo, folders, README (overview, problem statement, features, stack, structure, setup)	Architecture diagram, AI design section	Repo is public with a complete README and initial code
2	HTML structure	Home, Login, Register, Dashboard using semantic HTML5; nav links	Static lesson page with a placeholder video embed and tutor panel	All pages link to each other
3	CSS and responsive	External CSS, navbar, forms, cards, tables; Flexbox/Grid; breakpoints	Style the tutor panel and plan cards	No horizontal overflow at desktop, tablet, mobile
4	JS validation, dynamic nav	Validate login/register; show/hide Login, Register, Logout	-	Invalid forms cannot submit; nav reflects login state
5	JS modules and state	<code>auth.js</code> , <code>api.js</code> , <code>validation.js</code> , <code>ui.js</code> , <code>state.js</code> ; ES6 imports; browser storage	<code>ai.js</code> module; remember last-watched position in storage	Login/logout state works across pages
6	React and routing	Vite app; <code>/</code> , <code>/login</code> , <code>/register</code> , <code>/dashboard</code> , <code>/courses</code> ; Navbar, Footer, CourseCard etc.	<code>Lesson</code> , <code>StudyPlan</code> routes; <code>ChatTutor</code> , <code>VideoPlayer</code> components	All routes work with reusable components
7	Mock APIs and state	MSW or JSON Server; Axios; loading and error states; Context/Redux	Mock <code>/api/ai/*</code> and watch-event endpoints; <code>AIContext</code>	Dashboard shows student, courses, enrolment status from the mock API
8	Express backend	Node/Express, MVC folders, <code>.env</code> , CORS, JSON parsing, <code>GET /</code> and <code>GET /api/health</code>	<code>aiClient.js</code> stub	Server runs with clean structure
9	Student and course APIs	Register, login, students CRUD, courses CRUD, validation, status codes	Lessons, enrolment, progress, quiz endpoints	All endpoints pass in Postman
10	JWT and protected	Sign token on login, auth middleware, role middleware,	Forward identity and role to the AI service with a	Register, login, token, protected

#	Brief task	What you build	AI addition	Done when
	routes	401 on bad token, frontend logout	service token	access all work
11	Database schema	Users, Courses, Enrollments, Notifications (+ Progress); constraints; ER diagram	Lessons, watch events, quizzes, attempts, mastery, plans, content chunks	Schema documented and created
12	DB integration and CRUD	Mongoose models, DB errors handled, tested via Postman and frontend	Vector index on content chunks	Register, login, update, delete, and course CRUD all work on the real DB
—	<i>AI service build</i>	<i>(between Tasks 12 and 13)</i>	FastAPI, ingestion, tutor graph, mastery and planner, quiz generation, studio import, evaluation set	Tutor answers with citations; plan endpoint returns valid JSON
13	Socket.IO	Notifications, student chat, online/offline status, test with two browsers	Streamed tutor replies, AI-generated nudges	Second browser receives messages instantly
14	Security and errors	bcrypt, validation and sanitising, CORS, centralised errors, logging, env secrets	Prompt-injection guard, PII redaction, rate limits, cost logging	Bad input returns 400, not a crash
15	Deploy and test	Vercel (frontend), Render or equivalent (backend), production DB, env vars, full test pass, README with live URLs	Deploy the AI service; run evaluation on production	Live frontend URL, live API URL, working auth, CRUD, chat, and AI

12. Testing plan

- **API:** Postman collection (committed to `postman/`) covering every endpoint including 401/403/400 cases; Jest + Supertest for the critical paths.
- **Frontend:** validation behaviour, protected routes, and the lesson page (player events mocked).
- **AI service:** pytest for mastery maths, plan rules, schema validation, guardrails.
- **Real-time:** two browser windows: message delivery, notification delivery, tutor streaming.

- **Responsiveness:** desktop, tablet, mobile widths, no horizontal scroll.
- **Integration:** register → enrol → watch → quiz → plan generated → notification received.

13. Deployment and configuration

Piece	Where
React client	Vercel
Express API + Socket.IO	Render (or Railway)
FastAPI AI service	Azure Container Apps (fits an Azure OpenAI setup), or Render as a Docker service
Database	MongoDB Atlas
CI	GitHub Actions: lint, tests, build on every push

Environment variables (`.env.example`, never commit real values)

```
# server
PORT=5000
MONGODB_URI=
JWT_SECRET=
JWT_EXPIRES_IN=1d
CLIENT_ORIGIN=
AI_SERVICE_URL=
AI_SERVICE_TOKEN=
YOUTUBE_API_KEY=

# ai-service
AZURE_OPENAI_ENDPOINT=
AZURE_OPENAI_API_KEY=
AZURE_OPENAI_CHAT_DEPLOYMENT=
AZURE_OPENAI_EMBEDDING_DEPLOYMENT=
MONGODB_URI=
AI_SERVICE_TOKEN=

# client
VITE_API_URL=
VITE_SOCKET_URL=
```

14. Risks and decisions to check early

Risk	What to do
Some YouTube videos disable embedding	Test each imported video's embed; skip or flag ones that fail
Licence and attribution for NPTEL content	Check the licence terms on the NPTEL site; show attribution and a link to the original course on every lesson
Transcripts for the RAG tutor	Use transcripts/notes NPTEL provides for download or admin-uploaded text; avoid unofficial caption scrapers
YouTube Data API quota	Import a playlist once, cache the metadata in MongoDB
Azure OpenAI access or quota	Confirm your subscription can create the deployments early; keep the <code>11m.py</code> interface so a different provider can be swapped in
Free-tier cold starts (Render)	Warm up before demos; mention it in the README
Scope creep	Ship the core brief first, then the tutor, then the planner, then Studio and the admin analyst

15. Portfolio checklist

- ☐ Public GitHub repo with a clear README, architecture diagram and ER diagram
- ☐ Live frontend URL and live API URL in the README
- ☐ Evaluation results table (tutor accuracy, refusal accuracy, latency, cost)
- ☐ 2-3 minute demo video: enrol → watch → quiz → study plan → tutor answer → admin analytics
- ☐ Screenshots on desktop and mobile
- ☐ CI badge and a Postman collection
- ☐ A short write-up: "how the planner decides", and the trade-offs you made